# 5
# Beat Shazam

Now it's time to beat Shazam!

Like the previous chapter, this chapter focuses on applying Spring AI to another real-world use case.

I would assume most people who read this book know about the popular music app called Shazam. It can identify millions of songs in just seconds simply by listening to a small sample of music from a microphone, such as the mic on your smartphone. When Shazam identifies the song, it will tell you the song title and the name of the artist.

You can learn more about Shazam on Wikipedia at https://en.wikipedia.org/wiki/Shazam_(music_app).

I first stumbled upon Shazam when I was driving to work in 2011. I would be listening to music on the radio and wonder who the artist was and the title of the song. I thought, there must be an app that can identify songs simply by "listening" to music. After a quick search in the Apple App Store I found Shazam.

I never really thought much about how Shazam might be implemented until I started working with AI seriously in 2023. While on the Spring Data team, I also contributed to Spring AI. This sparked my interests in AI and thus began my journey. Even then, I really hadn't thought much about Shazam.

So, this is the topic that we will explore in this chapter. The main goal is to implement Shazam using Spring AI. Along the way, you will learn how to:

- Input and process audio using the Java Sound API
- Quantify sampled audio with Digital Signal Processing (DSP)
- Identify songs using audio fingerprints
- Use a Vector Store to store the generated Embeddings for a song
- Extend essential components of Spring AI to implement Shazam

These lessons are covered across the following sections starting with a:

- Backstory
- "Out of the sky, into the dirt"
- Implementing Shazam
- Processing audio with the Java Sound API
- Embedding audio using Spring AI
- Identifying a song using Spring AI
- Running Shazam

A full treatment of sound theory, digital signal processing, and implementing a robust version of Shazam is well beyond the scope of this chapter, or even this book for that matter. Hopefully, you will walk away with a better understanding of Shazam's inner workings and a deeper appreciation for its genius.

Let's get to it!

# Technical requirements

The Shazam application presented in this chapter is based on Java 17 and Spring Boot 3.5.x. The app uses Spring AI 1.0.x.

You must manually install JDK 17 or later. The project Maven POM declares and resolves Spring AI, Spring Boot and all required dependencies from Maven Central along with the JitPack IO Maven repository. You can find the source code for the Shazam application in the GitHub Repository for this book at:

https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/shazam-app

Additionally, you need to have Docker installed on your system. You can download Docker Desktop from docker.com/products/docker-desktop/. Docker is required to run the PostgreSQL, PGVector database used to store song metadata along with audio embeddings generated for the song.

After Docker is installed, there is nothing you need to do specifically to configure the Shazam application to connect to, configure and access the PGVector database. The `shazam-app` module contains a Docker `compose.yml` file along with `data.sql` and `schema.sql` DDL/DML SQL scripts to set up and initialize the database for the Shazam application.

Spring Boot provides built-in support for Docker Compose and processing Docker Compose YAML files. Refer to Spring Boot's documentation for additional details: docs.spring.io/spring-boot/3.5/how-to/docker-compose.html.

Finally, this example makes use of custom Spring AI extensions, which are covered in more detail in *Chapter 7*. While use of specific extensions is described in the context of the use case, please refer to *Chapter 7* for a complete description of available extensions and their functionality.

# Backstory

Disclaimer! While I may have blown a fuse or two implementing Shazam, no AI models were hurt during the process.

Feel free to skip ahead to the section on "Implementing Shazam" if you want to quickly get into the technical details. However, I share my thoughts and approach to solving the Shazam riddle in this section and the next. In particular, I share my application and system design using Spring AI.

After implementing the relatively simple example in **Chapter 2**, which introduced *cosine similarity search* to identify songs stored in a Vector database by lyrics, I thought, well, I could use a similar approach to build Shazam. This will be easy! Yeah right, okay.

The process seemed simple. I would use the Java Sound API to read in a song as sampled audio data (bytes) in MP3 format. Then, I would split the song into five-second audio clips.

The five-second measurement is not arbitrary. It takes Shazam an average of 2–5 seconds to identify a song. So, I figured, breaking an entire song into 5 second audio clips would be enough data to use in the search comparing stored audio clips with the snippet of audio sent by the user to the application when trying to identify a song.

However, it's possible for a user to record audio overlapping two audio clips. Therefore, I also created overlapping audio clips using the current and previous five-second audio clips.

Of course, this is still not an absolute guarantee to prevent overlap either, but it reduces the spread when recorded audio is approximately split across two audio clips evenly.

The following image is not precise, but splitting an entire song into 5 second audio clips looks roughly like the following diagram.

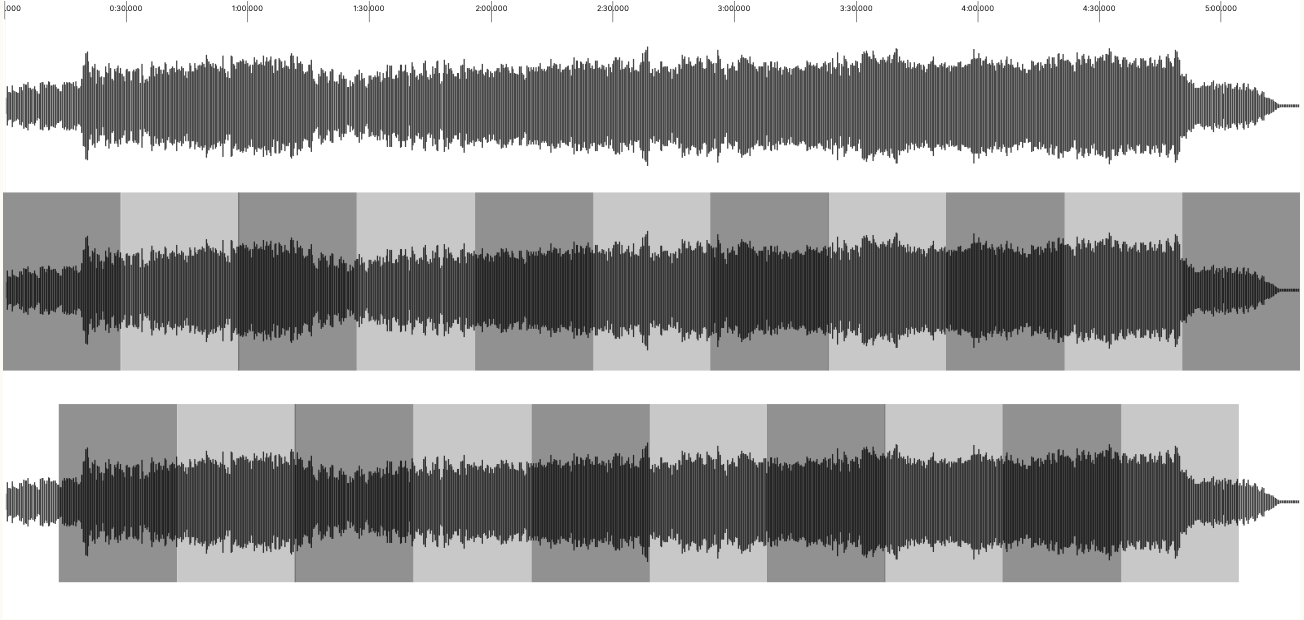


*Figure 5.1: Audio Split (not exact time intervals)*

Can you identify the song in the image above? Neither can I, but Shazam can. People who can identify a song from a waveform probably speak modem as well.

The first waveform is the entire song unchanged. The second waveform shows the song split into 5-second intervals. The last waveform shows overlapping 5-second audio clips between adjacent audio clips in sequence.

To be exact, the overlapping audio clip is the last 2.5 seconds of the previous audio clip combined with the first 2.5 seconds of the next audio clip adjacent to the previous audio clip. Remember, the algorithm is capturing and combining bytes of audio data that constitute the entire song.

---

NOTE

In my tests, I randomly saved an audio clip to manually verify that it was valid audio. I verified the audio by inspecting the headers using `ffprobe` and by listening to the audio clip. Additionally, I wrote an `AudioSplicer` that recombines all audio clips created by the `AudioSplitter` into a complete audio file to assert that the split and spliced audio were valid.

---

Additionally, my intent was also to limit how much audio the user could record and send to the app to 5 seconds.

Next, I would encode the bytes of audio data in Base64 format and create a Vector using an AI Embedding Model. The Vector would be stored as an Embedding in a Vector database. The audio bytes needed to be Base64 encoded, or encoded in some character encoding since Spring AI is limited to only embedding text documents.

Calling any implementation of `VectorStore.add(:List<Document>)` containing non-text-based `Documents` will throw an `IllegalArgumentException` stating, "*Only text documents are supported for now. One of the documents contains non-text content.*" So, we will need to work around this limitation.

I use the same process of splitting the audio data and generating `Embeddings` for both songs stored in the database as well as the audio clips sent by the user to the application for recognition. The beauty of storing only `Embeddings` for the audio of a song is that you don't infringe on the copyright. You cannot reconstitute a song from a series of `Embeddings`, or any other identifying form for that matter, as we will see later on.

Now, it is a simple matter to use the Vector database and run a *(cosine) similarity search* with the `Embeddings` generated from the audio sample sent by the user and the `Embeddings` of known songs stored in the Vector database used to identify the song.

The entire high-level song upload and identification process is depicted in the following diagram.

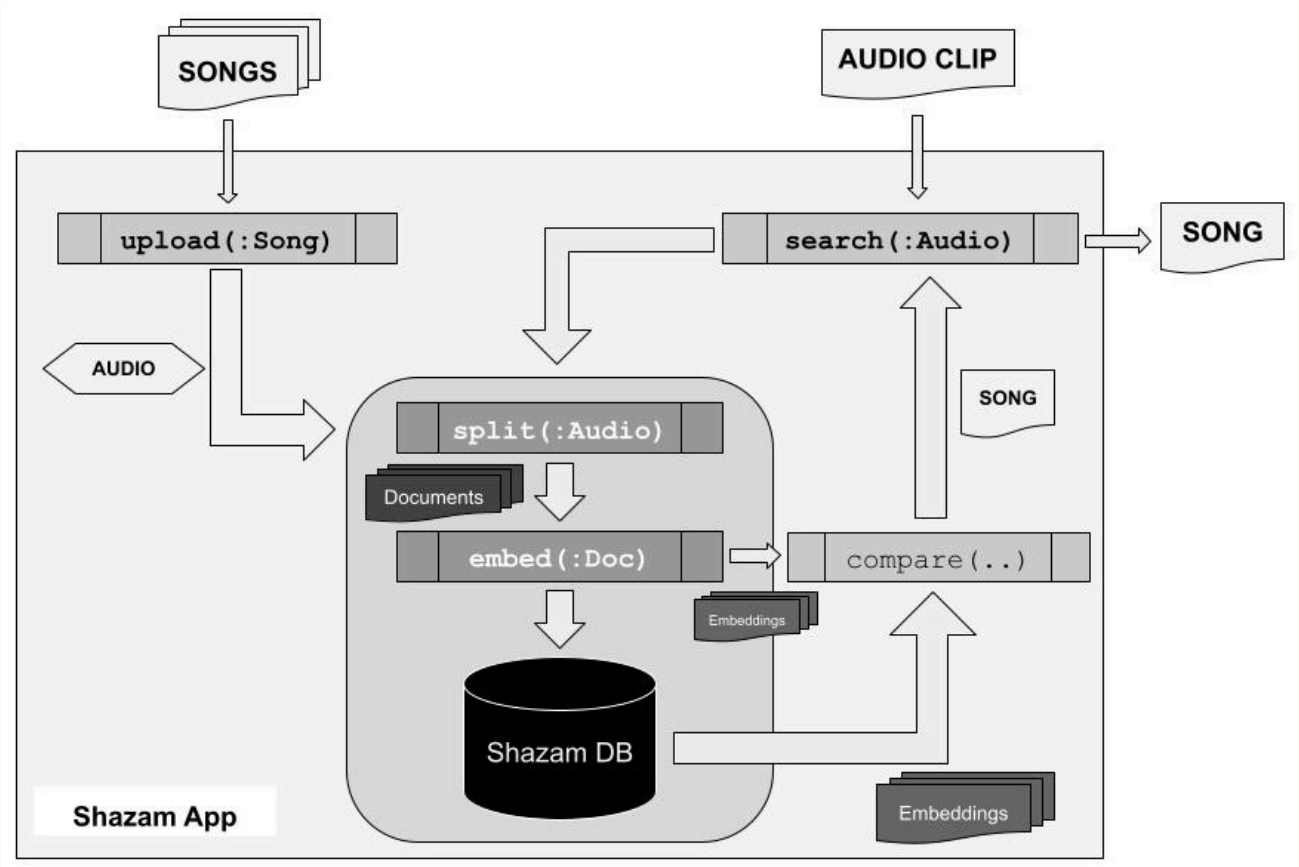



*Figure 5.2: Shazam App MusicService*

Unfortunately, parts of this implementation are severely flawed!

For one, you cannot simply Base64 encode audio bytes. The bytes of audio itself are meaningless. To make this concrete, imagine two different versions of the same song. We have the studio album version and a live concert version. While both versions of the song sound similar, for example have the same melody, both songs will have significantly different bytes of audio data.

This is when I turned to AI for help since it would seem to be a fitting use case.

## "Out of the sky, into the dirt"

Again, feel free to skip ahead to the section on "Implementing Shazam". This section covers several other approaches I attempted for context before I settled on building a custom Spring AI `EmbeddingModel` that uses audio fingerprinting.

If you followed the Wikipedia link on Shazam I shared at the beginning of this chapter, the journey for the original creators of Shazam began in 1999. AI was certainly a "thing" back then, but AI was not used to implement Shazam. Who knows what Apple did with Shazam after acquiring it in 2018 and what they have done with it since then.

In any case, I was still convinced that AI was a practical use case to build Shazam. So, I started searching for suitable AI models specifically trained to identify music when given sampled audio. Perhaps the most surprising limitation when implementing Shazam is that there really aren't any AI models readily available to identify music.

I searched across available AI providers, such as OpenAI, Google, Microsoft, Meta, etc. Of course, Apple wasn't going to publish any such AI model, even if it does exist. I searched local AI models available in Ollama and similar apps. I searched in Hugging Face. Still, nothing.

It turns out most of the audio-based AI models are predominantly trained for speech. Of course, there are also AI models used to generate amazing music as well.

This is when, at least for a short while anyway, I got the crazy notion of training an AI model myself. I started exploring *TensorFlow for Java* and *Deeplearning4j*. Ultimately, this is not a bad approach, but it certainly was not practical in the timeframe I gave myself to build Shazam. So, I tabled this in-depth research for later.

---

> You can learn more about *TensorFlow* for Java on GitHub at github.com/tensorflow/java. Learn more about *Deeplearning4j* on the project website at deeplearning4j.konduit.ai.

---

Hours and days went by. After finding several articles, a handful of code examples, and boring AI to tears with my insistent questions, I finally turned to audio fingerprinting.

---

NOTE

One of the more insightful articles I read was toptal.com/developers/algorithms/shazam-it-music-processing-fingerprinting-and-recognition. You can also learn more about Acoustic fingerprinting used to fingerprint sampled audio on the Wikipedia at wikipedia.org/wiki/Acoustic_fingerprint.

---

Acoustic fingerprinting led me to many other interesting sites.

I came across *MusicBrainz* (musicbrainz.org), which is when I learned about the *Acoustid* project (musicbrainz.org/doc/AcoustID). See also acoustid.org. The *Acoustid Chromaprint* software (acoustid.org/chromaprint) is behind the magic at *MusicBrainz Picard* to catalog and identify music.

The *Acoustid Chromaprint* library (github.com/acoustid/chromaprint) is written in C. Fortunately, if you scroll down in the GitHub repository project page, they list several implementations of the library available in other languages. One such implementation, *Chromaprint.scala* (github.com/mgdigital/Chromaprint.scala), written in Scala and therefore is applicable to the JVM.

Unfortunately, the binaries for *Chromaprint.scala* are no longer available and the GitHub project has been abandoned. The author has not worked on the Scala implementation in over seven years despite the fact that the source C library continues to be maintained, no doubt due to continued developments at *MusicBrainz*.

I was also unsuccessful in getting *Chromaprint.scala* to build out-of-the-box. I am working on a Maven port for its SBT-based build, which I should have available by the time this book goes to print.

Alternatively, I could use JNI, or JNA, to call the *Chromaprint* lib written in C from Java. This is when I turned to *Cursor* for help. I will just have *Cursor* re-implement the entire C implementation in Java. It nearly completed the task until I ran out of tokens. I suppose it would have been easier to generate the JNI/JNA code.

Fortunately, there is no shortage of audio fingerprinting libraries written in Java. The provider implementation I chose and adapted is not important. The important part was defining a Service Provider Interface (SPI) so I could easily swap implementations in case I encountered any limitations. The interface is covered below.

It's worth mentioning that you could implement an acoustic fingerprinting algorithm using a Digital Signal Processing (DSP) library. I found two really useful and robust libraries suitable for this task.

The first library I experimented with was TarsosDSP (github.com/JorenSix/TarsosDSP), which was developed by Dr. Joren Six while doing research on his Ph.D. thesis. The other library was jDSP (github.com/psambit9791/jdsp). During my journey, I found several other libraries as well, so take your pick.

It was all really fascinating, but time was of the essence. I wasn't interested in reinventing any wheels here. With my SPI firmly in place, I felt reassured and settled on an existing audio fingerprinting library.

While Shazam nearly beat me, this fight was far from over. Now it is time to implement Shazam!

# Implementing Shazam

The Shazam app is a full-featured Spring Boot Web application using Spring AI. A Spring Web MVC `@RestController` processes input from the user accessing the application through an HTML and JavaScript (jQuery) user interface on the front end. The UI indirectly interfaces with Spring AI on the backend through the `@RestController`.

The primary application service interface is the `MusicService`, which is defined as:

```
interface MusicService {
   // Search for a Song matching the given Audio
   Song search(Audio audio);
   // Store the Song and Audio to the database
   default void store(Song song) {
      store(song, (theSong, documents) -> documents);
   }
   void store(Song song, SongProcessor songProcessor);
}
```

Two main operations, `search`, and `store`, are all that are needed to implement Shazam. The Shazam application uses a database of existing songs to match and identify audio clips sent by the user to the application during the `search(..)` operation. The `store(..)` operation is used to pre-populate the database with songs that Shazam can match and identify.

The overloaded `store(..)` method accepts a `SongProcessor`, which is an extension of Java's `BiFunction` interface. The interface is used as a callback for callers that need to perform additional processing on the audio `Documents` created from the `Song`.

The two primary data types processed by the `MusicService`, and all downstream application components, are songs, and audio. To simplify the logic used to process songs and audio data, each were modeled with their own class.

The `Audio` class encapsulates the bytes of audio and is essentially defined as:

```
class Audio implements AudioSource, MediaSource {
  byte[] getData();
}
```

For convenience, the class also provides accessors for core Spring (AI) types, such as `Resource`, `Media` and `MultipartFile`, in addition to `java.io.File`. Optional metadata captures the `Duration` of the song and the Java Sound API `AudioFormat`. All of the metadata is used during processing and is captured at different stages in the audio processing pipeline.

Songs are modeled with the `Song` class to encapsulate the artist, album, and title of the song along with its audio data.

```
class Song implements AudioSource {
  Audio audio;
  String artist;
  String album;
  String title;
}
```

The core logic of the application is contained in the `SmartMusicService` class implementing the `MusicService` interface. A *Singleton* instance of the `SmartMusicService` delegates to other application components as necessary to perform the main functions of Shazam, namely `store` and `search`. Two of the most important application components are instances of the `AudioSplitter` interface and Spring AI's `VectorStore`.

The AudioSplitter is like Spring AI's TextSplitter used to chunk up large bodies of text, like entire documents. However, rather than splitting apart text, an AudioSplitter splits a stream of audio bytes into meaningful parts that can be compared for matching purposes. Splitting up audio is also essential to fit within the context window of the AI

model used to analyze audio data. Of course, the VectorStore is used to store embeddings generated from spliced audio in order to perform a similarity search when matching.

The `AudioSplitter` interface is defined as follows:

```
@FunctionalInterface
interface AudioSplitter {
    List<Document> split(Audio audio);
    default List<Document> split(AudioSource source) {
        return split(source.getAudio());
    }
}
```

The `AudioSplitter` interface is used to split up the audio for a song into 5-second audio clips that are stored in a List of Spring AI `Documents`. The implementation of this interface is built with Java Sound API and will be discussed later in the chapter.

When the `AudioSplitter` splits up the given `Audio` for a song, it stores the 5-second audio clips as instances of Spring AI's `Media` type in the `List` of Spring AI `Documents`. The audio clips are bytes containing a sequence of audio for the song. Spring AI `Documents` are needed to pass control to the `VectorStore` to generate `Embeddings` and perform `search` operations.

The Shazam app uses PostgreSQL's PGVector database to store both song metadata as well as the `Embeddings` generated from the audio clip `Documents` for the song. The entire logic for the `SmartMusicService.store(..)` method can be seen in the following code:

```
void store(Song song, SongProcessor songProcessor) {
    Assert.notNull(song, "Song is required");
    List<Document> documents = resolveSongProcessor(songProcessorprocess).(song);
    try {
        getVectorStoreaccept().(documents);
        getSongRepository().save(song);
    }
    finally {
        documents.forEach(getDocumentStore()::remove);
    }
}
```

The `SongProcessor` splits the audio for the song, the `VectorStore` is creating `Embeddings` for the audio clips, and the `Embeddings` along with the song metadata are stored in the PostgreSQL, PGVector database.

However, we have quite a lot to unpack from this neatly encapsulated snippet of code, which may appear to be overly simplified, but that was intentional. We will return to the `store(..)` method above shortly and talk a bit more about the `resolveSongProcessor(:SongProcessor)` method along with the use of a custom `DocumentStore`.

But first, let's start with the `SongProcessor` wrapping the `JavaSoundAudioSplitter` implementation that processes the audio for a song and prepares audio clips to be embedded using our `VectorStore` implementation, `PGVector`.

# Processing audio with the Java Sound API

The JDK includes the Java Sound API to process sampled audio. The API can also be used to control and affect the input and output of sound media, such as media from a Musical Instrument Digital Interface, or MIDI device. I use Java's Sound API for Shazam as a means to split the audio into discrete chunks, such as 5 second audio clips. The Java Sound API helps you with things like how many bytes constitute 5 seconds of audio given the sample rate, bits per sample along with other factors covered below.

While I cover the essential bits of the Java Sound API necessary to implement Shazam, a full treatment of the API is well beyond the scope of this book. You are encouraged to use available resources to learn more about Java Sound in general.

---

NOTE

You can learn more about the Java Sound API from the source (oracle.com/java/technologies/java-sound-api.html). Additionally, Oracle provides a *programmers guide* (docs.oracle.com/javase/8/docs/technotes/guides/sound/programmer_guide/contents.html) and *tutorial* (docs.oracle.com/javase/tutorial/sound/index.html) to demonstrate the capabilities of the API as well as to help familiarize yourselves with how to use it. You will also find other useful resources in the provided links.

---

Initially, I started working with MP3 audio files. The primary motivation to use MP3 is that audio files are widely available and compressed. Compression makes the task of transferring audio over the web from frontend to backend cheaper.

By comparison, Pearl Jam's "Alive" song is a 5 minute, 41 second, 6.8 MB MP3 file. The same song is 60.2 MB as a WAV file.

One of the factors in MP3 compression is that it removes frequencies we as humans cannot hear. Humans hear between 20 Hz and 20,000 Hz. Audio recorded to a CD is 44,100 Hz, or 44.1 kHz. Removing these non-perceivable frequencies makes no difference to us. The audio sounds the same. However, don't try to convince your dog of that. Dogs can hear up to 60 kHz, so he might just bark at you!

Out of the box, the Java Sound system does not support MP3. A quick test on my Mac running Oracle JDK 17.0.18 for macOS ARM reveals the following support for audio formats: AIFF, AU and WAVE. MP3 is an add-on, or "plugin", as it is referred to.

> TIP
>
> The `shazam-app` Maven module from the example repository for this book contains supporting Java programs to analyze audio files and log properties of the Java Sound system, like the supported audio file formats mentioned above. These programs can be found in the `com.packt.spring.ai.examples.app.shazam.support` package. The `SupportedAudioFileFormatsApp` was used to log supported audio file formats.

When reviewing the resource links I shared in the note above, you will find a reference to the Tritonus Java library (tritonus.org). This library contains codecs for the MP3 format, which allow Java programs to read, record, and play MP3 audio. It is also a provider implementation of the Java Sound API.

Since Java Sound, circa 2000, predates many modern technologies, coming by the bits is not immediately automatic. Fortunately, Google published the bits to Maven Central. The following dependency can be declared in your Maven POM to acquire the bits:

```
<dependency>
  <groupId>com.googlecode.soundlibs</groupId>
  <artifactId>tritonus-all</artifactId>
  <version>0.3.7.2</version>
</dependency>
```

Unfortunately, the published library only lets you write MP3, not read it. Therefore, I had to download the `Tritonus MP3 JAR` specifically and declare a system dependency since it is not available in Maven Central. The dependency declaration in the Maven POM appears as follows:

```
<dependency>
  <groupId>org.tritonus</groupId>
  <artifactId>tritonus_mp3</artifactId>
  <version>0.3.6</version>
```

```
    <scope>system</scope>
    <systemPath>${project.basedir}/libs/tritonus_mp3-0.3.6.jar</systemPath>
  </dependency>
```

You will quickly discover that the code in the `tritonous_mp3` JAR depends on JLayer (github.com/umjammer/jlayer), another library and MP3 decoder implemented in Java. Google published this library to Maven Central as well, but the version is out of date. Alternatively, you can acquire the latest bits from JitPack.io using the following repository and dependency declarations:

```
  <repository>
    <id>jitpack-io</id>
    <name>JitPack IO Repository</name>
    <url>https://jitpack.io</url>
  </repository>
  <dependency>
    <groupId>com.github.umjammer</groupId>
    <artifactId>jlayer</artifactId>
    <version>1.0.3</version>
  </dependency>
```

After much wrangling, I eventually got everything working, but ultimately decided to switch from MP3 to WAVE. I put the required bits for MP3 support in the "mp3" Maven Profile of the Shazam app's Maven POM for interested readers.

The main reason for choosing WAVE over MP3 is that audio metadata pulled from an MP3 file using the Tritonus Java Sound library with extensions is incomplete. This is apparent in the output from the `AudioFormatResolverApp`:

`Audio format [MPEG2L3 22050.0 Hz,` **`unknown`** `bits per sample, stereo,` **`unknown`** `frame size,` **`unknown`** `frame rate]`

Not surprisingly, "unknown" bits of information aren't really all that helpful when processing audio. This is when I turned to the FFmpeg project for additional help.

> tip
>
> Additional information for an audio file can be obtained using the `ffprobe` program (ffmpeg.org/ffprobe.html) from the FFmpeg project. For convenience, I implemented an API wrapper around the `ffprobe` program, encapsulated in the `FFProbe` class. The details of this class are not important. It is only used internally and is not something users need to concern themselves with while using the Shazam app.

Two of the most important bits of metadata needed when splitting up the audio data are sample rate, or frame rate, and frame size.

Of course, it is possible to calculate the frame rate and frame size if you know the size of the audio file in bytes and its duration. But, this was always the challenge with MP3 metadata. MP3 file header information was always missing, and it required the help of external programs, like FFmpeg/ffprobe. This made WAVE ideal since the metadata was always present in the audio files I used.

Sample rate and frame rate are the same. It measures the number of samples, or frames, captured in a given second of audio. For audio recorded to a CD, the sample/frame rate equals the hertz. Therefore, the number of samples/frames per second of audio is 44,100. For an MP3, it is half of that at 22,050.

Sample size is typically 16 bits. You can think of sample size as the resolution of the digital sound wave. Frame size is 2 times sample size since audio is recorded in stereo using 2 channels. When combined, you can determine how many bytes are taken up by 1 second of audio, or in our case, 5 seconds.

With this information, it is now possible to split up the audio for a song. The algorithm used in the `JavaSoundAudioSplitter` implementation appears below:

```
try (AudioInputStream in = openInputStream(audio)) {
  AudioFormat audioFormat = in.getFormat();
  Duration audioClipDuration = getAudioProperties().getClipDuration();
  List<Document> documents = new ArrayList<>();
  long frameLength = computeFrameLength(audioFormat, audioClipDuration);

  while (in.available() > 0) {
    // read clips (chunks) of audio and store in a Document
  }

  return documents;
}
```

First, a Java Sound API `AudioInputStream` is opened to read all the bytes of audio from the song. Using the `AudioFormat` from the stream and a configured `Duration` for the audio clip, for example, 5 seconds, we can now compute the frame length of the audio clip. The frame length is the total number of frames of audio data to capture from the source audio stream of the song in 5 seconds.

NOTE

Using a configuration property, `shazam.audio.clip.duration`, from `application.properties`, allows us to tune the amount of audio captured per audio clip. Therefore, we can use 5 seconds, 1 second, 100 ms, or whatever is necessary.

---

To properly capture the exact number of audio bytes from the source audio stream and create an audio clip based on the configured `Duration`, we must open a sub-`AudioInputStream` limited by the frame length. Then, the audio bytes are subsequently written to the `ByteArrayOutputStream` using the Java Sound `AudioSystem`.

This code inside the `while` loop shown above, used to capture the exact number of audio bytes and store them in a `Document`, is shown below:

```
try (AudioInputStream clipIn = openInputStream(in, audioFormat, frameLength)) {
  ByteArrayOutputStream out =
      new ByteArrayOutputStream(clipIn.available());

  AudioSystem.write(clipIn, AUDIO_FILE_TYPE, out);

  byte[] audioData = out.toByteArray();

  AudioClip audioClip = AudioClip.from(audioData, audioFormat)
      .atTimestamp(timestamp)
      .fromByteOffset(byteOffset);

  Document audioDocument = buildDocument(audioClip);
  documents.add(audioDocument);

  // ...
}
```

The Java Sound `AudioSystem` must be used to write the bytes for the audio clip so its contents are actual audio data with valid header information. With proper audio metadata, the audio clip is simply a partial sequence of bytes from the source audio stream of the song.

From the audio bytes, we construct an `AudioClip` that also captures the timestamp in the source audio stream marking the start of the audio clip along with the byte offset. The `AudioClip` is then encapsulated in a Spring AI `Document` and added to a `List` of `Documents` for all audio clips that make up the entire song.

This `List` of newly created Spring AI `Documents` containing the audio clips is then returned to the `SmartMusicService`. In the previous section, I mentioned the `resolveSongProcessor(..)` method. In this method, the `SmartMusicService` adds an internal `SongProcessor` to associate the given `Song` with the `List` of `Documents` containing the audio clips for the `Song`.

The `SongDocument` association is added to the property metadata of the `Document`, which gets stored in the `VectorStore` as JSON. The logic for creating the association is pretty straightforward and appears as follows:

```
private Document associateSong(Document document, Song song) {
  document.getMetadata().put(SONG_ID_KEY, song.getId());
  return document;
}
```

Now, if a `Document` is matched during a search, we will have a key to the `Song`. This is the only way to make an association between application-controlled model types and Spring AI types, such as `Document`.

The next step is to hand off the `Documents` to be processed and stored by the `VectorStore`, which we review in more detail in the next section.

# Embedding audio using Spring AI

The next significant application component in the audio processing pipeline is the `VectorStore`. The `VectorStore` is used to store `Embeddings` generated from `Documents` containing audio clips for a song. Technically, the actual work is performed by a configured AI `EmbeddingModel` implementation used by the `VectorStore` to generate `Embeddings`.

As we discussed in **chapter 2**, Spring AI includes support for many `VectorStore` providers, such as PGVector, and AI `EmbeddingModel` implementations. All Spring AI `EmbeddingModel` implementations are backed by AI models that currently generate embeddings for textual data. Although our data is not text, it is bytes of audio. Attempting to generate embeddings for non-textual documents leads to an `IllegalArgumentException` as mentioned previously. This limitation requires us to customize Spring AI in order to process different media types other than text, such as audio.

Since all Spring AI's VectorStore implementations falsely assume Spring AI EmbeddingModel implementations only generate Embeddings from textual Documents,

the "contents" of a Document are retrieved as text. This is apparent from Spring AI's AbstractObservationVectorStore base implementation, from which all VectorStore implementations extend, to validate that all Documents passed to the add(:List<Document) method are text only, as shown below:

```
void add(List<Document> documents) {
 // Throws IllegalArgumentException for non-text Documents
 validateNonTextDocuments(documents);

 // Embed each Document and store it in this VectorStore
}
```

It is also apparent in a VectorStore provider implementation, such as PGVector, which always extracts the contents of a Document as text.

```
var content = document.getText();
```

As a result, we must trick Spring AI into thinking our audio media-typed `Documents` contain text, and not audio bytes. We do this by storing the `Document` ID as its "content" and caching the `Document` in-memory. Then, the `Document` content, or its ID, is used to look up the `Document` in the cache before it is processed by our custom AI `EmbeddingModel`. Of course, our custom AI `EmbeddingModel` knows to use the cache to retrieve the `Document` containing the audio.

To temporarily cache the `Documents` containing audio rather than text, the SmartMusicService uses a `SongProcessor`. This is where the `DocumentStore` comes into play. This custom interface defines an API to cache `Documents` using typical cache operations like get and put. The default implementation simply caches `Documents` in memory keyed by ID.

The custom AI `EmbeddingModel` knows how to use the `DocumentStore` (cache) to look up the audio `Documents`. The `VectorStore`'s only job is to use the custom configured `EmbeddingModel` to generate `Embeddings` using AI from the audio content of the `Document` before it is stored.

You have no immediate control over the interface between the `VectorStore` and `EmbeddingModel` encapsulating the AI used to generate the `Embeddings`, hence the use of the `DocumentStore` to perform a lookup. Once the `Documents` are processed, the cache is cleared of those `Documents`.

That was a lot to digest. The logic described above can be represented in a sequence diagram shown below:

Though not depicted in Figure 5.3 above, the last step in the audio processing pipeline is simply to save the metadata of the song to the database. Once again, the `VectorStore` and SQL database are one and the same. This makes managing all the data required by Shazam easy to index and reference.

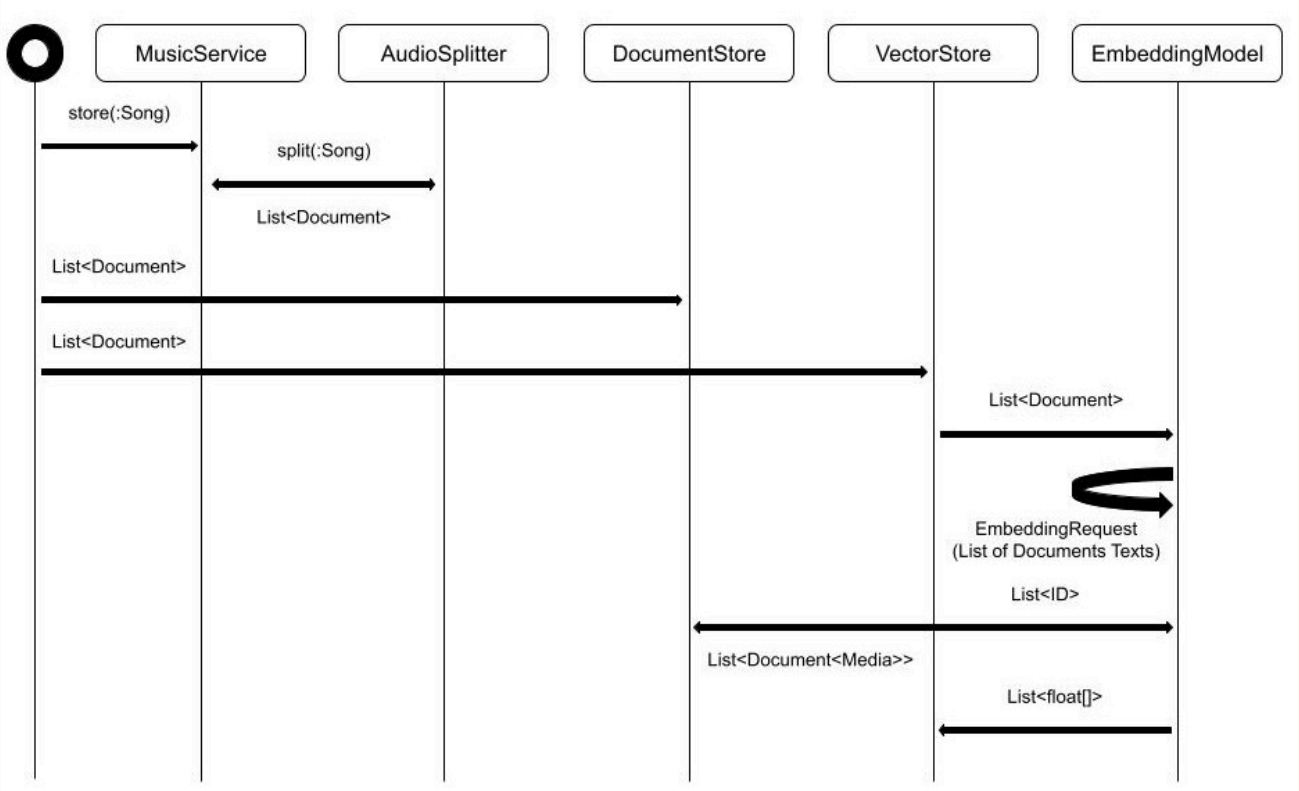



*Figure 5.3: Storing a Song*

Behind all the application component interactions is where the real magic happens;: generating `Embeddings` for all audio clips that make up the entire song. Attempting to generate `Embeddings` from audio data using an actual AI `EmbeddingModel` was the main AI play, until it wasn't.

The `EmbeddingModel` is not just generating `Embeddings` from audio clips. It is generating `Embeddings` from digital fingerprints computed from audio clips. As a result, generating `Embeddings` for even small amounts of audio data, such as 100 ms, is really expensive given the sheer volume of digital fingerprints to embed.

For example, to compute digital fingerprints and generate embeddings for a 5 minute, 41 second song, not exceeding the context window and with consideration for other factors, takes over 5 hours. This latency is not practical for use in any application.

Technically, computing fingerprints is fast, even for an entire song. On average, the configured audio fingerprinting library generated 136,500 fingerprints in ~2500 ms for the 5 minute, 41 second song. The latency comes from interacting with an `EmbeddingModel` over HTTP through the `EmbeddingModel's` RESTful interface combined with the sheer volume of fingerprints to embed.

---

NOTE

For testing purposes, I primarily used the `nomic-embed-text` and `qwen3-embedding` AI models running locally, in Ollama.

---

I think the `EmbeddingModel` approach with AI warrants further exploration in the future as AI models continue to evolve. This will be specifically applicable to AI models trained for audio recognition or AI models that can create `Embeddings` from audio data directly.

Using provided client drivers, rather than a RESTful interface, is also an important consideration. Until then, we just have to wait.

Regardless of this limitation, the example Shazam app was designed as if it were using AI. To the application, there is no difference. Plugging in an AI model later on will have no impact on the `MusicServicestore(:Song)` and `search(:Audio)` operations. This is because the `MusicService` delegates to a `VectorStore`, which uses an `EmbeddingModel`, and both are Spring AI interfaces defining a proper SPI.

As mentioned above, I chose an existing SQL/ `VectorStore` provider implementation, PGVector. For `EmbeddingModel`, I created the custom `AudioEmbeddingModel` class to encapsulate the logic for generating embeddings from audio. This allows the implementations to be easily replaced, such as using an actual `EmbeddingModel` backed by an AI model.

The function of the `AudioEmbeddingModel` class is no different than any other provider implementation, such as `OllamaEmbeddingModel` or `OpenAiEmbeddingModel`. The overridden `embed(:Document)` method generates an `Embedding` from text in order to quantify the data semantically. In this case, it computes audio fingerprints from the given audio and then embeds the computed fingerprints, returning `Embeddings`, or floating-point vectors.

The implementation of `AudioEmbeddingModel` appears in the code below.

```
class AudioEmbeddingModel extends AbstractEmbeddingModel {
    @Override
    @NonNull float[] embed(@NonNull Document document) {
```

```
        Audio audio = DocumentUtils.toAudio(document);
        Fingerprint<?> audioFingerprint = getAudioFingerprintFunction().compute(audio);
        Embedding embedding = getAudioFingerprintEmbeddingFunction().embed(audioFingerprint);
        return embedding.getOutput();
    }
}
```

The `AudioEmbeddingModel` enlists the help of two Java-based `Functions`, `AudioFingerprintFunction` and `AudioFingerprintEmbeddingFunction`. Both `Functions` enable the library used to compute audio fingerprints as well as the algorithm that generates `Embeddings` from an audio fingerprint to be exchanged.

The `Function` definitions are self-explanatory. The `AudioFingerprintFunction` is defined as:

```
interface AudioFingerprintFunction<T> extends Function<Audio, Fingerprint<T>> {
    @Override
    default Fingerprint<T> apply(Audio audio) {
        return compute(audio);
    }
    Fingerprint<T> compute(Audio audio);
}
```

And the AudioFingerprintEmbeddingFunction is defined as:

```
interface AudioFingerprintEmbeddingFunction extends Function<Fingerprint<?>, Embedding> {
    @Override
    default Embedding apply(Fingerprint<?> audioFingerprint) {
        return embed(audioFingerprint);
    }
    Embedding embed(Fingerprint<?> audioFingerprint);
}
```

The `AudioFingerprintFunction` accepts `Audio` and computes a `Fingerprint`. In turn, the `AudioFingerprintEmbeddingFunction` accepts the computed `Fingerprint` and produces an `Embedding`. The `Embedding.getOutput()` method returns the resulting floating-point vector that is stored in the `VectorStore`.

Additionally, an audio/digital fingerprint has been modeled with the generically parameterized `Fingerprint` interface, defined as:

```
interface Fingerprint<T> {
   T get();
   default byte[] getData() {
    throw new UnsupportedOperationException("...");
   }
   default void consume(Consumer<T> consumer) {
   }
   default String toHexString() {
      // ...
   }
}
```

The `consume(:Consumer<T>)` method enables a custom `Consumer` object to process the raw digital fingerprint data without coupling callers to the raw digital fingerprint data, which varies by provider implementation. In the case of audio fingerprints, the component type `T` is a list of hashed spectrogram peaks corresponding to timestamped frequency features that are discriminative for audio identification.

The abstract `Fingerprint` data type and combination of `Functions` enable different provider implementations to be easily interchanged without impacting the overall procedure of the `AudioEmbeddingModel.embed(..)` method. Overall, this makes experimentation and testing efforts more flexible, which was very helpful during discovery and implementation phases when building Shazam.

---

NOTE

The design could have been altered so that Spring AI `Documents` were created from audio fingerprints rather than audio bytes. This may simplify the interaction between the `VectorStore` and `EmbeddingModel` and would eliminate the need for the `DocumentStore` lookup. However, if an AI `EmbeddingModel` were available to accept audio directly, then it nullifies the need to compute audio fingerprints manually. Also, digital fingerprints aren't the only way to identify audio.

---

A detailed discussion covering the technical implementation of audio fingerprinting algorithms is beyond the scope of this chapter. There are many implementations readily available. As mentioned above, the Acoustid project's Chromaprint library is one of the more popular implementations in use. Plus, by using a robust **Digital Signal Processing** (**DSP**) library, such as **TarsosDSP** or **jDSP**, it is possible to develop your own implementation.

---

NOTE

I am concurrently working on a side-project to port the Chromaprint C library to Java to complement the Shazam app built for this book. Cursor is

helping with this effort. I am also porting the Chromaprint.scala project SBT build to Maven. And I am considering the use of JNI or JNA to provide a simple native interface in Java. An Adapter would better enable the code to stay in sync with the original C implementation, which is the source of truth. Interested readers can track this effort in the following GitHub repo: github.com/jxblum/chromaprint/tree/main

---

Now that we have the ability to split a song into audio clips, embed the audio clips as a set of audio fingerprints that identify the original song, and then store the Embeddings in our VectorStore, it is time to turn our attention to search. The Shazam implementation must use the stored Embeddings when running a similarity search to identify a user-recorded audio clip from a song.

# Identifying a song using Spring AI

Searching and identifying a song is the last mile in our Shazam application implementation.

We have already done the hard work of digitizing and identifying music. Now, we simply rinse and repeat. Except, instead of storing audio, we match the sampled, recorded audio of a song sent by a user to an identified song stored in our pre-populated database. If there is a match, then we simply return the song title and artist. Otherwise, Shazam simply doesn't know.

It should be apparent by now that Shazam is only as good as the songs it stores in the database. However, you might imagine that one day, Shazam could still be able to tell you the artist of a song simply by listening to the acoustics, such as the artist's voice, the instrumental, and the features of a song.

The actual matching algorithm used by Shazam is not a vector database cosine similarity search. However, for the Shazam implementation presented in this chapter, that is precisely what the application uses.

An important factor in matching the audio for a song is the similarity threshold applied by the vector database when performing the similarity search. The similarity threshold for the Shazam application is controlled by the following property defined in `application.properties`:

```
shazam.song.search.similarity-threshold=0.85
```

When this property is set to 0.85, as shown above, an 85% match is required to identify the song when comparing vectors generated from the audio clip sent by the user to the vectors of the songs stored in the database.

Of course, it is not quite as simple as just setting the "similarity-threshold."

Similar songs, even by different artists, can match in sound and frequency. Additionally, as you might imagine, different songs by the same artist will most likely have similar frequencies and sounds. The latter is not all that surprising since songs from the same artist will have similar beats, rhythms, amplitudes, frequencies, and so on, given the genre of music.

This is where the timestamp capturing when a frequency occurred during the song becomes important. Of course, the same frequency can occur several times throughout the song, and even in different songs by different artists. So, it is a set of frequencies, spaced apart by time but close together in succession, that match the desired song.

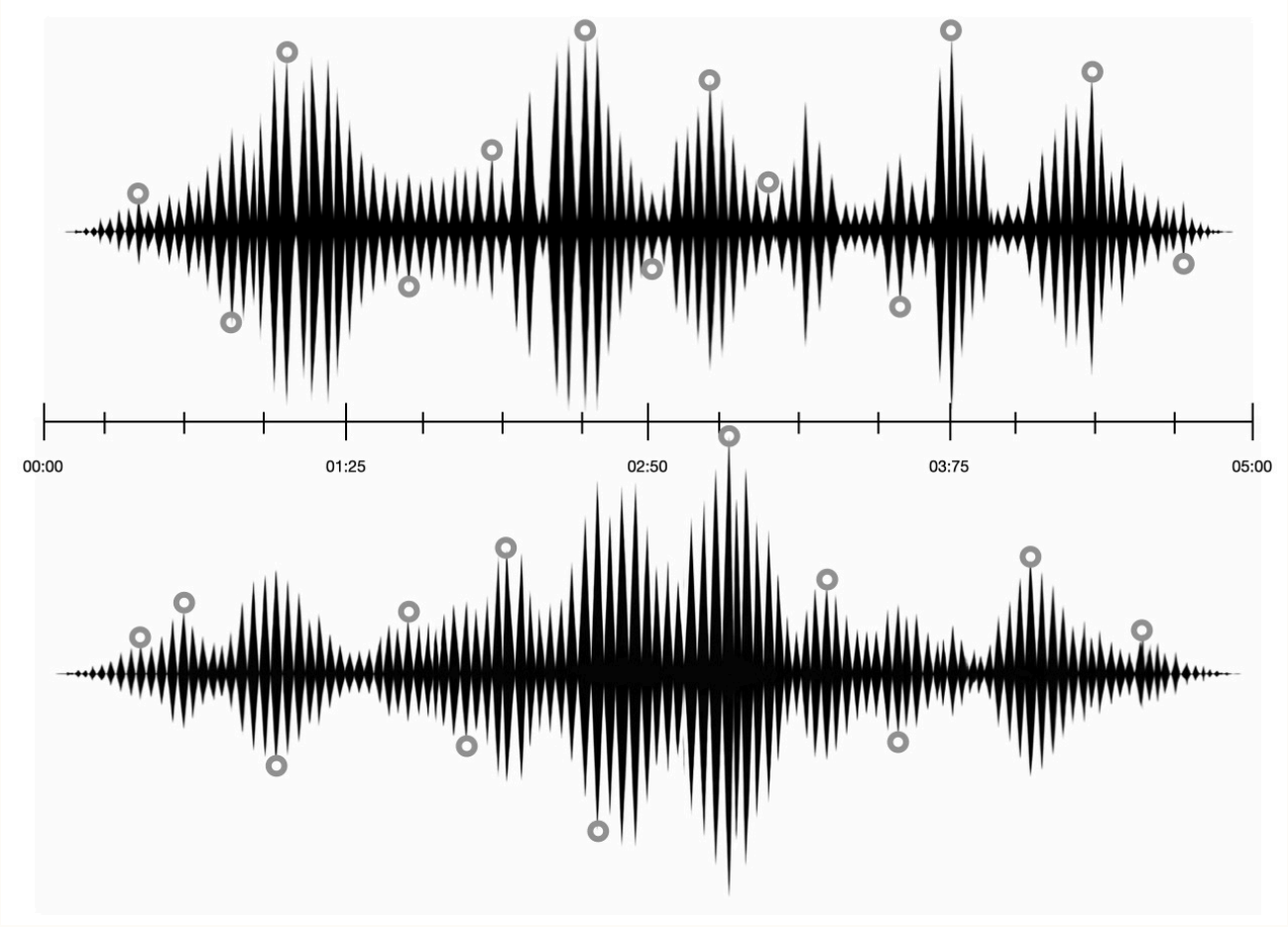



*Figure 5.4: Frequencies of 2 Sound Waves*

The diagram above shows frequencies of sound waves along a timescale for two different songs. Key features of the sound waves are selected by the audio fingerprinting algorithm and represented by the gray circle dots.

NOTE

I say "loosely depicted here" because there are far more data points captured during audio fingerprinting.

---

If the following audio clip is captured from a media input device, such as your smartphone, and sent to the Shazam app, then it can match the series of sound features from the original audio as seen in the following diagram:

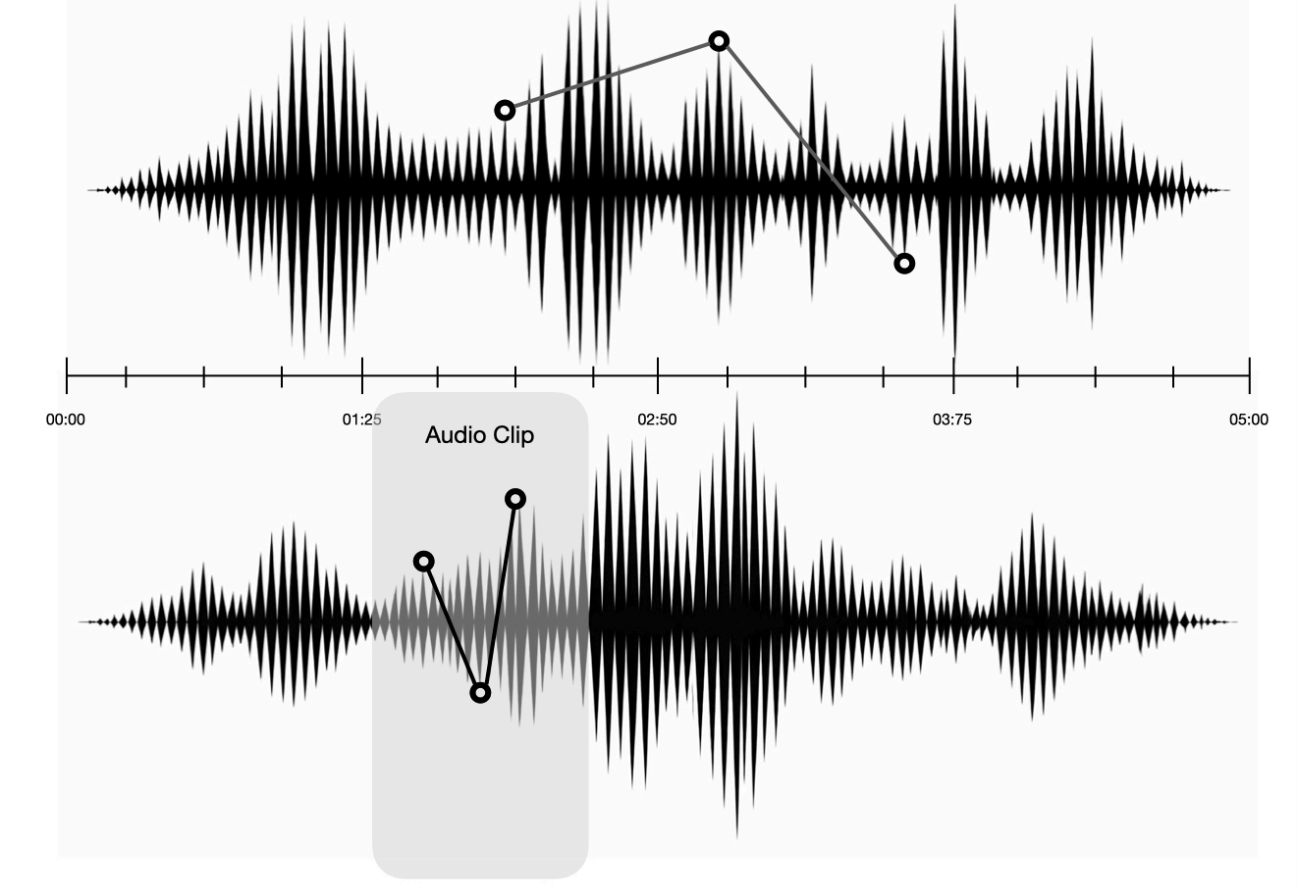



*Figure 5.5: Frequencies of Audio Clip*

While sound waves from 2 different songs can have the same frequencies, when matching a sound wave from an audio clip to a song, it is important that the matching frequencies occur in sequence. Thus, you can measure how far apart key frequencies occur relative to each other, which should match the same set of frequencies captured from the matching song. These key frequencies are features of the sound wave captured during the audio fingerprinting process.

This logic is used by the `MusiService.search(:Audio)->Song` method to match Audio to a `Song`. The method is shown below:

```
Song search(Audio audio) {
  AudioSearchRequest searchRequest = buildSearchRequest(audio);
  Document document = searchRequest.toDocument();
  try {
```

```
    getDocumentStore().save(document);
    List<Document> matchingSongs = getVectorStore().similaritySearch(searchRequest);
    ImmutableSet<UUID> songIds = identifySongs(matchingSongs);
    if (songIds.isNotEmpty()) {
      UUID matchingSongId = getSafely(songIds::onlyOne, e -> {
        throw new NonUniqueSongException(songIds, e);
      });
      return getSongRepository().findById(matchingSongId)
        .orElseThrow(() -> new SongNotFoundException(..));
    }
    throw new SongNotFoundException("Song not found");
  }
  finally {
    getDocumentStore().remove(document);
  }
}
```

The first step is to build a Spring AI `SearchRequest` object from the given `Audio` clip recorded and sent by the user to the application. In this case, the `MusicService` builds a custom `AudioSearchRequest` extending the `SearchRequest` class to hold the `Audio` data.

The implementation of the `buildSearchRequest(:Audio)` method is shown below:

```
AudioSearchRequest buildSearchRequest(Audio audio) {
  SearchRequest searchRequest = SearchRequest.builder()
    .similarityThreshold(songSearchProperties.resolveSimilarityThreshold())
    .topK(songSearchProperties.resolveTopK())
    .query(UuidGenerator.INSTANCE.generateId())
    .build();
  return AudioSearchRequest.builder(searchRequest)
    .query(audio)
    .build();
}
```

An `AudioSearchRequest` is built from an instance of `SearchRequest` configured with the similarity threshold required to make a match and top K limiting the number of matches returned. The `Audio` included in the `AudioSearchRequest` object is used in the query of the similarity search after computing audio fingerprints and generating embeddings.

The newly constructed `AudioSearchRequest` object gets passed to the `VectorStore` when running the similarity search to match the audio to audio clips of songs stored in

the `VectorStore`.

```
getVectorStore().similaritySearch(searchRequest);
```

The audio processing used to generate audio fingerprints and Embeddings from the audio clip sent by the user during search is exactly the same as the audio processing used to upload and prepopulate the database with songs.

After all matching `Documents` are found, it is time to identify the `Song`.

```
ImmutableSet<UUID> songIds = identifySongs(matchingSongs);
```

If you recall, we added the `Song` ID to the metadata of the `Documents` containing the audio clips for a song when vectorized and stored to the database. Additional matching logic may be necessary to further filter potential song matches. This is where Shazam's algorithm comes into play, and the timestamp must be considered. This is also why breaking the song into a sequence of audio clips is important, precisely so you can capture the timestamp in which a range of audio features occur.

This logic is encapsulated in the `identifySong(..)` method given the `List` of matching `Documents` containing audio data returned from the similarity search. Then it is a simple matter to assert that one and only one song by ID ultimately matches before the song metadata is looked up from the database:

```
getSongRepository().findById(matchingSongId)
```

The `SongRepository.findById(..)` method will look familiar to readers who are familiar with Spring Data. The `SongRepository` is an extension of Spring Data's `CrudRepository` interface used to perform CRUD and basic query operations for `Songs` stored in the database.

There is little reason why the `findById(..)` method would return a `nullSong` entity at this step in the code and throw a `SongNotFoundException` unless the "matching" `Song` was deleted prior to search. Of course, if there is no matching `Song`, a `SoundNotFoundException` is thrown and the user will be notified in the app.

Now that the implementation is complete, it is time to run Shazam.

# Running Shazam

The Spring Boot application is all set up to run Shazam out of the box.

Running the Shazam Spring Boot application is as simple as running the following Maven command from the command line:

```
spring-ai-examples> mvnw --projects shazam-app spring-boot:run
```

The Maven project takes care of bootstrapping the entire application, including the PostgreSQL PGVector database, compliments of the Docker `compose.yml` file and configuration of the `spring.docker.compose.file` property in `application.properties`. All you need is to install Docker Desktop. Maven and Spring take care of the rest.

Once the application starts, your Web browser will automatically open to the Spring Boot application. The application appears as follows:

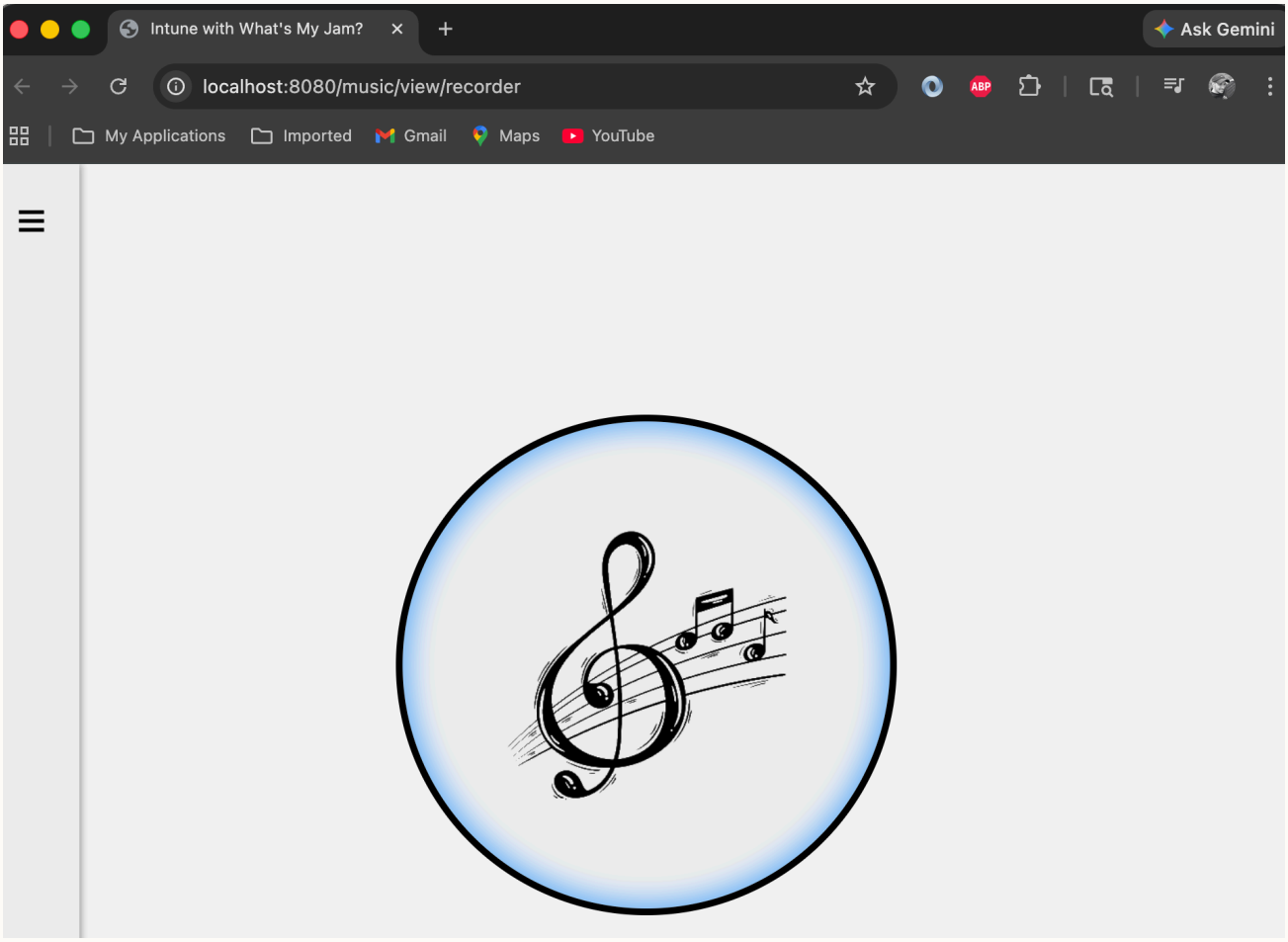



*Figure 5.6: Shazam app*

Click on the music note button to activate your system's mic and start recording an audio clip. Simply play music through your smartphone or other audio device to have the Shazam app try to identify a song.

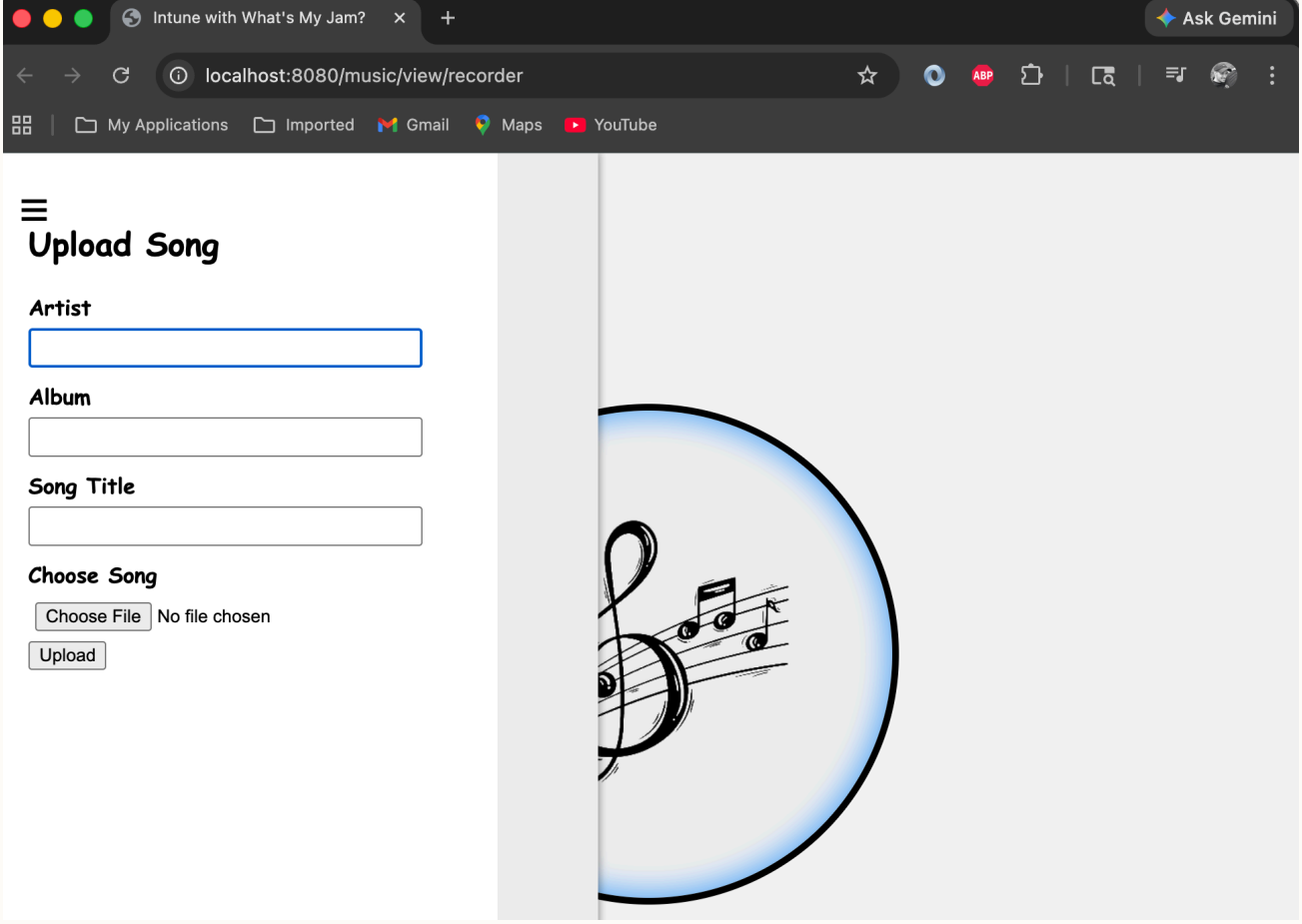



*Figure 5.7: Upload Song*

Alternatively, I also created the `SongLoader` Spring Boot application to upload songs from WAVE files contained in the `src/main/resources/database/data` directory. This is useful for bulk loading songs in batches rather than individually using the application UI.

The WAVE files must be accompanied by a metadata file containing the song title, artist, and album. The metadata filename must match the song filename.

The database data directory listing appears as follows:

```
$ ll shazam-app/src/main/resources/database/data
total 124048
-rw-r--r--@ 1 jxblum  staff      181 Mar  8 18:06 Matchbox20-Unwell-metadata.json
-rw-r--r--@ 1 jxblum  staff  2287671 Apr  8  2004 Matchbox20-Unwell.mp3
...
```

The contents of a metadata file are in JSON format. The metadata file for Pearl Jam's song, Jeremy, from the Ten album appears as follows:

```
{
  "artist": "Pearl Jam",
```

```
    "album": "Ten",
    "title": "Jeremy",
    "year": 1991,
    "resourcePath": "classpath:/database/data/PearlJam-Ten-Jeremy.wav"
  }
```

Only "`artist`" and "`title`" are the required metadata for a song. The "`resourcePath`" is used to locate the audio for the song if the audio filename does not match the metadata filename.

For convenience, every song contained in the `/src/main/resources/database/data` directory is automatically loaded by the `SongLoader` Spring Boot application by default.

Congratulations, you can now Shazam music!

# Summary

In this chapter we saw how Spring AI was extended to implement a complex use case, such as Shazam. It was no small or easy feat, and there is much room for improvement, but it is possible.

The most important bit is to capture meaningful information from sampled audio. This applies to both songs uploaded, stored, and managed in our Shazam database for recognition as well as to audio clips recorded by a user and sent to the Shazam app for identification.

Audio was processed using the Java Sound API. Songs were split into 5 second audio clips, which is the same duration permitted by the app when allowing users to record and send audio for search. Digital fingerprints were computed from the sampled audio in order to capture and timestamp discriminating sound features that are used to match audio to songs. To simplify the identification process, embeddings were generated from the fingerprints and stored in the `VectorStore`, which could later be used in the similarity search.

All of this logic was neatly encapsulated in a custom `AudioEmbeddingModel` with the intent that, once an AI model is readily available for identifying music, the `EmbeddingModel` provider implementation can be easily changed. Changing `EmbeddingModel` implementations would have no impact on the design of the application.

Additionally, different `AudioFingerprintFunction` implementations could be used to change audio fingerprinting algorithms or even strategies, such as implementing a fingerprinting algorithm using TarsosDSP or jDSP.

In any case, the Shazam app demonstrates the versatility of Spring AI's powerful APIs to adapt to any use case. Regardless of whether Spring AI was built for such use cases out of the box, Spring's adherence to the Open/Closed principle makes these use cases possible and should not be underestimated.

By the way, the song in the waveform from figure 5.1 is Pearl Jam's song, Jeremy, from the Ten studio album. Keep on rockin', rockers!

In the next chapter, you are going to learn how Spring AI can be used to call multiple AI models in the same application. You will observe and learn how different AI models can interact with each other while competing in a friendly game of Connect 4.


## Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

*Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.*

Previous chapter

< Chapter 4: Real-Time Message Translation

Next chapter

Chapter 6: Connecting the Dots >